

Deep Learning (AI2100/AI5100/CS5480)

Assignment–2

Deadline: 20th April, 2026

Submission: (A2-<roll-number>.ipynb)

Total: 200 Marks

Please read the following instructions before starting the assignment:

- It is **strongly recommended** that you work on this homework on an individual basis. If you have any questions or concerns, feel free to contact the instructor or the TAs at least 2 days before deadline.
 - You can also use markdown cells in the notebook to explain theoretical problems and observations.
 - Ensure that all submitted code runs without errors. Non-executable code will receive zero marks for the corresponding question.
-

1 Numpy Implementation of CNN

(15 Marks)

In this assignment, you will implement each of the components of a Convolutional Neural Network (CNN) from scratch (i.e., without using built-in functions for convolution, pooling, non-linearity, padding, or striding). Your implementation must accept an image input and generate an output vector. Use random weights for filter kernels and fully connected layers.

- Convolution function:** It accepts as input an image, a filter kernel, stride, padding, and the non-linear function. The function must correlate (convolve) the input image (after padding if specified) with the kernel (at the specified stride size) and generate an output activation after applying the specified non-linearity. Verify with standard non-linear activation functions such as sigmoid, tanh, ReLU, and Parametric ReLU (PReLU). Display the input image, the filter kernel, and the output activation map. Ensure that your function can accept multi-channel input and a corresponding kernel volume. (3)
- Pooling function:** It accepts as input the activation map output from the convolution function, a pooling function, and stride. The function must output the appropriately pooled activation map. Display the input activation map and the pooled output. (2)
- Convolution layer function:** It accepts as input a volume (image or activation maps), filter kernels, stride, padding, and the non-linear function. The function must convolve the input volume (after padding if specified) with each of the kernels (at the specified stride size) and generate an output activation volume after applying the specified non-linearity. Display the input image or activation maps, filter kernels, and output activation maps. Verify that the output has the expected size ($W \times H \times C$). (1)
- Pooling layer function:** It accepts as input the activation map volume, the pooling function, and stride, and generates a pooled output volume. Display the input and output volumes. (1)
- Flattening function:** It accepts as input the activation map volume output by the pooling layer and generates a vector of a specified size. Note that this function has a weight matrix associated with it whose size must match the input and desired output sizes. (1)

- vi) **Multilayer Perceptron (MLP):** It accepts as input a vector, the number of hidden layers, the size of each hidden layer, the non-linear function, and the size of the output layer. Generate the output vector both with and without applying the softmax function at the output layer. (2)
- vii) **Feed-forward CNN architecture:** Use the above functions to implement a CNN with the following architecture:

- Input image of size $32 \times 32 \times 3$ (CIFAR-10 dataset)
- Convolution layer with 4 kernels of size 5×5 , ReLU activation, stride 1, and same spatial dimensions as input
- Max pooling layer of size 2×2 with stride 2
- Convolution layer with 4 kernels of size $5 \times 5 \times 4$, ReLU activation, stride 1, and same spatial dimensions
- Max pooling layer of size 2×2 with stride 2
- Flattening layer (identity mapping)
- MLP with one hidden layer of 49 nodes and an output layer of 10 nodes
- ReLU activation and softmax output

Verify that the network accepts an image input and outputs a vector of appropriate dimension. (5)

2 PyTorch Implementation of CNN (25 Marks)

- a) Train the CNN described in Question 1 (part vii) for image classification using PyTorch. Initialize all network parameters randomly. (2)
- b) Use cross-entropy loss to compute the error at the softmax output layer. Note that the ground-truth labels are one-hot encoded vectors of dimension 10. (9)
- i) Vanilla SGD with learning rate $\eta = 0.001$.
- ii) Momentum with $\eta = 0.001$, $\alpha = 0.9$.
- iii) RMSProp with $\eta = 0.001$, $\rho = 0.9$.

For training:

- Choose 100 images per class from the training set (total 1000 images).
 - Experiment with different mini-batch sizes.
 - Shuffle the training data after each epoch.
 - Train for 15 epochs (you may experiment with more epochs).
- c) Perform the following after each epoch: (5)
- i) Compute the error on training and test datasets. Use 10 images per class from the test set (total 100 images). Plot training and test error as a function of epochs.
- ii) Visualize activation maps by selecting representative slices from the activation volumes of each convolution layer.
- iii) Report the classification accuracy.
- d) Use tSNE to visualize the bottleneck feature at the end of the first epoch and the last epoch. Use the same set of test images as in the previous question. (3)

- e) Apply dropout at the hidden layer of the MLP with dropout rates $p = 0.2, 0.5, 0.8$. Train using RMSProp and report results. Compare performance with the model without dropout and comment on differences in training and testing performance. (3)
- f) Introduce batch normalization (BN) at the hidden layer of the MLP and report the performance. Batch normalization should be applied at the output of the hidden layer before applying the non-linearity. In your implementation, include both BN and dropout (with $p = 0.2$). Compare the performance with the vanilla implementation as well as the model with only dropout, and comment on the training and testing performance. (3)

3 Backpropagation in CNN (Theory)

(20 marks)

Consider the convolutional neural network described below. The input is an 8×8 grayscale image X .

- A 3×3 convolutional filter with bias, stride = 1, and padding = 1 is applied to produce feature map Z_1 .
- A ReLU activation is applied to obtain A_1 .
- A 2×2 max-pooling layer with stride = 2 is applied to obtain P_1 .
- A second 3×3 convolutional layer with **2 filters**, stride = 1, and no padding is applied on P_1 , producing Z_2 .
- A ReLU activation is applied to obtain A_2 .
- The feature maps are flattened into a vector F , which is passed through a fully connected layer (with weights W , bias b) to produce Z_3 .
- A Sigmoid activation is applied to obtain the final output $\hat{y}$.

The network is trained using binary cross-entropy (BCE) loss.

Answer the following:

- (i) Draw a clear flow diagram of the network described above. (2)

- (ii) Determine the dimensions of the following: (3)

$$Z_1, \quad A_1, \quad P_1, \quad Z_2, \quad A_2, \quad F, \quad Z_3, \quad \hat{y}$$

- (iii) Compute the total number of trainable parameters in the network. (3)

- (iv) Using backpropagation, derive the gradients of the loss with respect to: (2+4+6)

- Fully connected layer parameters (W, b)
- Second convolutional layer parameters
- First convolutional layer parameters

4 Building Sequence Learners from Scratch: Vanilla RNNs, BPTT, and LSTMs (70 marks)

In this Question, you will build sequence learning models entirely from scratch using only standard Python and NumPy. You will implement a Vanilla Recurrent Neural Network (RNN) and a Long Short-Term Memory (LSTM) network to solve a "many-to-one" sequence task, such as Sentiment Analysis. You will code the forward passes, derive and implement Backpropagation Through Time (BPTT), and perform ablation studies to understand the internal mechanisms, vanishing gradients, and the effects of architectural variations.

Part 0: Dataset, Input Representation, and Evaluation Protocol

To ensure consistency across implementations, all students must follow the specifications below.

Dataset: You will use the [IMDb Movie Reviews Dataset](#) for binary sentiment classification. You are **not required** to use the full dataset. You can do the experiments using a subset of the dataset as well.

- Each sample consists of a movie review (a sequence of words) and a label:
 - $y = 1$ (positive sentiment)
 - $y = 0$ (negative sentiment)
- You may use a preprocessed version of the dataset (tokenized and cleaned), but you must clearly document all preprocessing steps.
- Split the dataset into training, validation, and test sets (e.g., 80/10/10).

Input Representation: Since RNNs operate on numerical vectors, you must convert text into vector representations. You have to use both of the following approaches and report any differences that you may find:

1. One-Hot Encoding:

- Represent each word x_t as a one-hot vector of size $|V|$, where V is the vocabulary.
- This leads to high-dimensional sparse inputs.

2. Learned Word Embeddings (Recommended):

- Initialize an embedding matrix $E \in \mathbb{R}^{|V| \times d}$.
- Each word is mapped to a dense vector:

$$x_t = E[w_t]$$

where w_t is the index of the word at time step t .

- The embedding matrix should be trained jointly with the model.

Sequence Handling:

- Fix a maximum sequence length T (e.g., $T = 50$ or $T = 100$).
- Truncate longer sequences and pad shorter ones with a special token.

Evaluation Metrics: To evaluate model performance, report the following metrics on the **test set**:

1. Accuracy

2. F1-Score

Reporting Requirements:

- Report Accuracy and F1-score for both:
 - Vanilla RNN
 - LSTM
- Compare results across:
 - Different sequence lengths
 - Optimizers (SGD vs Adam)
- Include loss curves and discuss convergence behavior.

4.1 Part 1: Vanilla RNN and the Forward Pass (12)

Your first task is to implement the standard Recurrent Neural Network architecture. RNNs are designed to handle variable-length inputs by applying the same transformation at every time step, modeling the dependence among the inputs.

Tasks:

1. Initialize the hidden state at $t = 0$ as $h_0 = 0$. (2)

2. Implement the recurrent state update for a sequence of inputs x_t : (7)

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$$

3. Implement the output layer for a many-to-one configuration (e.g., predicting the final sentiment at the end of the sequence): (3)

$$y_t = \text{softmax}(W_{hy} \cdot h_t + b_y)$$

4.2 Part 2: Backpropagation Through Time (BPTT) and Gradient Issues (18)

To train the network, you must implement BPTT. The error is a scalar function of the sequence, and the computation shares the same parameters across time, meaning you must pool the gradients.

Tasks:

1. Implement the BPTT algorithm. You will need to compute gradients $\frac{\partial L}{\partial V}$, $\frac{\partial L}{\partial W}$, and $\frac{\partial L}{\partial U}$. (6)

2. Code the gradient calculation for the shared weight W accumulating over time steps. Use the chain rule formulation provided in the lectures for a sequence of length T : (6)

$$\frac{\partial L}{\partial W} = \sum_{k=1}^T \frac{\partial L}{\partial h_T} \left(\prod_{j=k+1}^T \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W}$$

3. **Exploding Gradients:** Because the product $\prod_{j=k+1}^T \frac{\partial h_j}{\partial h_{j-1}}$ can lead to exploding gradients, implement **Gradient Clipping** to prevent your loss from returning NaN values. (6)

4.3 Part 3: Long Short-Term Memory (LSTM) Implementation (20)

Vanilla RNNs struggle to learn long-term dependencies because the gradient vanishes when the product of the derivatives $\frac{\partial h_j}{\partial h_{j-1}}$ shrinks over long sequences. To fix this, you will implement an LSTM, which utilizes a cell state C_t to store long-term information and gates to erase, write, and read information.

Tasks: Implement the LSTM node and its gates, ensuring all relevant outputs are bounded between $[0, 1]$ using a sigmoid (σ) function where appropriate. Use the concatenated vector $[h_{t-1}, x_t]$ for your computations.

1. **Forget Gate:** Decide what to throw away from the cell state. (2)

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. **Input Gate & Candidate State:** Decide what new information to store. (4)

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

3. **Cell State Update:** Combine the forget and input gates to update the memory. (6)

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

4. **Output Gate & Hidden State:** Compute the output and the next hidden state. (8)

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

(Hint: Verify during your backward pass that the gradient $\frac{\partial L}{\partial h_t}$ does not vanish because $\frac{\partial h_t}{\partial C_t}$ depends on a diagonal matrix $D(o_t \odot \sigma'(C_t))$, allowing gradients to flow unimpeded through the cell state C_t).

4.4 Part 4: Ablation Studies (20)

Once you have built both models from scratch, run the following ablations to demonstrate your understanding of the architectures and optimizers.

Ablation A: The Vanishing Gradient in Action (5)

- Experiment with sequence lengths of $T = 5, 20$, and 50 .
- Plot the magnitude of the gradient $\frac{\partial L}{\partial h_1}$ for both the Vanilla RNN and the LSTM.
- **Question:** Explain your observations based on the RNN gradient flow equation. Why does the gradient vanish in the Vanilla RNN but not the LSTM?

Ablation B: Coupled Input and Forget Gates (10)

- Modify your LSTM implementation to use **Coupled Input and Forget Gates**.
- Instead of computing an independent input gate i_t , force the network to only input new information when it forgets old information.
- Replace your standard Cell State update with:

$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

- **Question:** Compare the training time (epochs to convergence) and final accuracy of the standard LSTM vs. the Coupled Gate variant. Does reducing the parameter count improve or hurt performance on your dataset?

Ablation C: Optimization Strategies (5)

- Train your networks using standard **Stochastic Gradient Descent (SGD)**, where $w = w - \eta \nabla_w L(w, x_i, y_i)$.
- Train your networks using the **Adam** optimizer, ensuring you implement the bias correction step since the estimates start from 0.
- **Question:** Plot the loss curves. How does Adam compare to standard SGD when optimizing the complex loss surface of an LSTM?

5 Building a Transformer Encoder from Scratch: Attention, Training, and Analysis (40 marks)

In this question, you will implement the core components of a Transformer encoder using only Python and NumPy. You will build a self-attention mechanism, construct a single-layer Transformer encoder, and apply it to a sequence classification task (e.g., sentiment classification). You will also analyze attention behavior and optimization.

Part 0: Dataset and Setup

You will use the [IMDb Movie Reviews Dataset](#) for binary sentiment classification. You are **not required** to use the full dataset. You can do the experiments using a subset of the dataset as well.

- Tokenize text and map words to indices.

Input Representation:

- Use learned embeddings $E \in \mathbb{R}^{|V| \times d}$.
- Add **positional encodings** to retain order information.

5.1 Part 1: Scaled Dot-Product Attention (10)

Implement the core attention mechanism.

Tasks:

1. Compute queries, keys, and values:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

2. Implement scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

3. Verify that attention weights sum to 1 across each row.

5.2 Part 2: Multi-Head Attention (5)

1. Split Q, K, V into h heads.
2. Compute attention independently for each head.
3. Concatenate outputs and apply a final linear projection.

5.3 Part 3: Transformer Encoder Block (10)

Build a single Transformer encoder layer.

Components:

- Multi-head attention
- Feedforward network:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- Residual connections
- Layer normalization

Task:

- Implement a full forward pass of the encoder block.

5.4 Part 4: Sequence Classification (4)

- Use the final representation of a special [CLS] token or mean-pooling.
- Pass it through a linear layer + softmax for classification.

5.5 Part 5: Training and Optimization (6)

- Train using cross-entropy loss.
- Implement:
 - SGD
 - Adam (with bias correction)
- Plot loss curves.

5.6 Part 6: Analysis and Ablations (5)

Ablation A: Effect of Number of Heads

- Compare $h = 1$ vs $h = 4$
- Report accuracy and training behavior

Ablation B: Positional Encoding

- Remove positional encoding
- Explain performance change

6 Long-Context Modeling: From RNNs to Selective State Space Models (30 marks)

We consider a modified version of the IMDB dataset called **IMDb-Long**. To ensure consistency and eliminate label leakage, you must construct the IMDb-Long dataset as follows:

Neutral Text Source (Strictly Neutral)

Use the **WikiText-2** dataset:

- `datasets.load_dataset("wikitext", "wikitext-2-raw-v1")`

Extract sentences from WikiText-2 as neutral filler.

Rationale:

- Wikipedia text is largely factual and topic-diverse
- It is independent of IMDb sentiment labels
- It contains minimal sentiment bias compared to reviews

IMDb-Long Construction

For each IMDb review:

1. Tokenize the review and split it into three parts:

$$\text{review} = (P_1, P_2, P_3)$$

2. Discard P_2 .
3. Sample random sentences from WikiText-2 and use them as filler.
4. Construct the final sequence:

$$\text{final} = P_1 \parallel \text{neutral filler} \parallel P_3$$

5. Append additional neutral sentences if needed to reach target length:
 - Experiment with each of 256, 512, or 1024 tokens.
6. Keep the original IMDb label unchanged.

Important Constraints

- Neutral filler must come **only** from WikiText-2
- Do not reuse IMDb sentences as filler
- Ensure tokenizer is identical across all models
- You may use whatever subset of IMDb you used for **Q5**
- Use fixed random seed for reproducibility

6.1 Part 1: Computational Scaling (5)

Let the sequence length be L and hidden dimension be d .

- Derive the time complexity per layer of a Transformer encoder using self-attention. (2)
- Derive the per-sequence time complexity of a standard RNN. (2)
- For $L = 1024$, $d = 256$, compute the ratio: $\frac{\text{Transformer cost}}{\text{RNN cost}}$ (1)

6.2 Part 2: Gradient Propagation in RNNs (5)

Consider the recurrence:

$$h_t = Ah_{t-1} + Bx_t$$

- Express h_t in terms of h_0 and $\{x_1, \dots, x_t\}$. (2)
- Show that:

$$\frac{\partial h_t}{\partial h_{t-k}} = A^k \quad (2)$$

- Let $\rho(A)$ be the spectral radius: (1)
 - What happens if $\rho(A) > 1$?
 - What happens if $\rho(A) < 1$?

6.3 Part 3: State Space Models (SSM) (6)

Consider the discrete SSM:

$$h_k = Ah_{k-1} + Bx_k, \quad y_k = Ch_k$$

- Show that:

$$y_k = \sum_{i=0}^k CA^{k-i}Bx_i \quad (3)$$

- Interpret this as a 1D convolution with kernel:

$$K_{k-i} = CA^{k-i}B \quad (2)$$

- Explain why this system is **time-invariant** and state one limitation. (1)

6.4 Part 4: Selective State Space Models (6)

We now introduce a simple selective mechanism.

$$h_k = Ah_{k-1} + Bx_k$$

$$y_k = g_k \odot (Ch_k), \quad g_k = \sigma(W_g x_k)$$

- Verify that all dimensions are consistent. (2)

- Explain why this model is no longer time-invariant. (2)

- Show that:

$$y_k = g_k \odot \left(\sum_{i=0}^k CA^{k-i}Bx_i \right)$$

Explain why this cannot be represented as a fixed convolution. (2)

6.5 Part 5: Implementation and Experiments (8)

You must implement and compare the following models:

- LSTM
- Transformer Encoder
- Linear SSM
- Selective SSM

6.5.1 (a) Linear SSM Implementation

Complete the following:

```
1 class LinearSSM(nn.Module):
2     def __init__(self, d_in, d_hidden, num_classes):
3         super().__init__()
4         self.A = nn.Parameter(torch.randn(d_hidden, d_hidden) * 0.1)
5         self.B = nn.Linear(d_in, d_hidden)
6         self.C = nn.Linear(d_hidden, d_hidden)
7         self.fc = nn.Linear(d_hidden, num_classes)
8
9     def forward(self, x):
10        batch, seq_len, _ = x.shape
11
12        # TODO: initialize hidden state
13        h = None
14
15        for t in range(seq_len):
16            x_t = x[:, t]
17
18            # TODO: recurrence
19            # h = A h + B x_t
20            pass
21
22        # TODO: output
23        y = None
24        out = None
25        return out
```

6.5.2 (b) Selective SSM Implementation

```
1 class SelectiveSSM(nn.Module):
2     def __init__(self, d_in, d_hidden, num_classes):
3         super().__init__()
4         self.A = nn.Parameter(torch.randn(d_hidden, d_hidden) * 0.1)
5         self.B = nn.Linear(d_in, d_hidden)
6         self.C = nn.Linear(d_hidden, d_hidden)
7
8         # TODO: define gating network
9         self.W_g = None
10
11        self.fc = nn.Linear(d_hidden, num_classes)
12
13    def forward(self, x):
14        batch, seq_len, _ = x.shape
15
16        # TODO: initialize hidden state
17        h = None
18
19        for t in range(seq_len):
20            x_t = x[:, t]
21
22            # TODO: update state
23            # h = ...
24
```

```

25         # TODO: compute gate
26         g = None
27
28         # TODO: selective output
29         y = None
30
31     # TODO: classification
32     out = None
33     return out

```

6.5.3 (c) Experiments

Train all models for sequence lengths {256, 512, 1024}.

Report:

- Accuracy
- Training time per epoch

Plot:

- Accuracy vs sequence length

(d) Analysis

Answer the following:

- Which model degrades most with longer sequences?
- Does Selective SSM outperform LSTM or Transformer?
- Give one failure case of Transformer on long sequences.